Development

As a SaaS provider, our customers expect us to maintain the confidentiality and integrity of their data, as well as the reliability and availability of the application their teachers and students depend on. To meet those expectations, we have designed a policy that documents requirements for management and developers to follow throughout the software development lifecycle.

Coding Standards

Input Validation

Apply the most restrictive input validation to all customer-inputted data fields that doesn't rule out plausible input. Client-side validation errors should provide user-friendly inline prompts informing the user of acceptable data. Server-side validation errors should throw exceptions. Well-vetted data input libraries should be used.

Hardcoded Access Credentials

Access Credentials (whether usernames, passwords, or keys) should never be stored in source code or configuration files. Access credentials should only be stored in a secure external repository (e.g., AWS Secrets).

Credentials for accessing SQL in the development environment is an exception to this rule and are allowed in a developer's local configuration.

Logging Handler

All logging should be made through the central handler, and not through any other means (e.g., written to the local disk or emailed). Explicit consideration should be given to the nature of the log entry and tagged as informational, warning, or error as appropriate.

Sensitive information should never be logged (e.g., passwords, access keys, AWS secrets, credit card data, or NPPI).

Exception Handling

All code must be covered by a default handler that captures unhandled exceptions, logs them through the central logging handler, and prevents exposure of details to the end-user (e.g., stack traces and raw error messages).

Access Control

For *every* retrieval, modification, or insertion of customer data, a determination must be made as to whether or not the current user is entitled to the action. So for every piece of logic designed to retrieve, modify, or insert data an accompanying piece of logic must be designed to answer the question: will we allow the current user to do this?

No new data access logic may be implemented without this accompanying access entitlement validation.

Injection Defense

Inputted customer data can never be trusted and must always be treated as potentially malicious. Customer data concatenated with other text to form commands is a particularly dangerous scenario. Some scenarios of concern:

1. SQL queries
2. JSON, YAML, or XML serving as input to an API.
3. URL's to APIs with parameters in query strings.

In each of these examples, a command formed from concatenated strings is vulnerable to attack if the concatenation includes user-inputted strings. If those user-inputted strings are syntax-aware and malicious by design, they can potentially break out-of-band and deliver their own SQL, JSON, shell commands, etc.

To avoid this threat, always use available higher level operations that bypass the need for string-concatenation in the first place.

For SQL, use parameterized queries or ORM.

For XML and JSON, use .NET's serialization/deserialization functions.

For API URLs, use appropriate string escape libraries.

For YAML a 3rd party serializer should be considered.

Never used shelled commands.

- Customer uploaded files should always be written to S3.
- Web code should never write anything to the local machine.

Output Sanitization

All data (whether or not user-inputted) outputted to the client may potentially break out-of-band from its containing HTML, CSS, or Javascript and so must be properly escaped.

External API Calls

All calls to external APIs must be over secure TLS connections.

Vetted Modules

Critical security algorithms dealing with cryptography should be implemented with standardized widely accepted, regularly reviewed, scrutinized, and updated modules (such as those included in the OS or .NET). They should not be implemented manually.

Library Lifecycle

We use the Long Term Support release of .NET, along with supported versions of 3rd party .NET libraries and NuGet packages.

We use React on a release no older than one major release behind.

Code Reviews

All pull requests require review by a member of the applicable approval group prior to merger. The three approval groups are frontend, backend, and database. Frontend react code must be approved by a member of the frontend group. Backend C# code must be approved by a member of the backend group. All SQL changes must be approved by a member of the database group. Scripts not subject to pull requests also need to be reviewed.

Web Application Vulnerability Scanning

On a monthly basis the cloud administrator and CTO review the results of the web application vulnerability report. For each finding we make a determination as to applicability (i.e., does it actually pose a security risk in our environment). For inapplicable findings, we accept them in the scanning application to prevent future reports. For applicable findings we create Jira tasks, assigning priority as commensurate with the risk posed.

Vulnerability Reporting

All development personnel should remain vigilant to threats to the confidentiality/integrity of customer or company data, or to the reliability/availability of customer-facing or internal applications/systems. Upon detection of such vulnerabilities, the employee/contractor will alert the CTO who will prioritize the threat, and assign remedial tasks as appropriate, tracking them in Jira.

Vulnerability Remediation

Our policy is to remediate critical vulnerabilities as soon as possible. This is usually within 2 weeks, depending on our release schedule, the vulnerability and competing priorities.

For high vulnerabilities, our policy is to remediate in less than 60 days.

Vulnerability Root Cause Analysis

For every vulnerability remediated, a root cause analysis will be performed. This will involve a determination as to the deficiency that caused the vulnerability to arise in the first place. Root causes may be related to (but not limited to) the following: (1) inadequate design, (2) inadequate training, (3) inadequate security/access controls, (4) unreliable 3rd parties. Upon determining the root cause, development will work with the CTO to map out, prioritize, and assign tasks to prevent this type of vulnerability in the future.

Personally Identifiable Information

The Personally Identifiable Information policy applies.

Security Training

All developers will complete the SANS Developer Security Training annually.

Policy Exception Provision

The CTO reserves the right to make exceptions to policies as unanticipated needs arise.